

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Course Code:	CSA12	Course Title:	Computer Architecture for Multicore Systems
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)
Weightage:	30%	Total Marks:	25
CO5 Statement:	<i>Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.</i>		
Student Name:		Reg.No.:	
Department:		Date:	

ANNEXURE A
Pseudocode Writing Task (Algorithm Design)

1. Write pseudocode to design a DMA-based data transfer algorithm that moves data from an I/O device to memory while handling interrupts and minimizing CPU involvement.

Ans:

ALGORITHM DMA_Data_Transfer

BEGIN

 Initialize DMA Controller

 Initialize I/O Device

 Set Memory Address ← Starting Memory Location

 Set Transfer Size ← Required Data Size

 Configure DMA Controller:

 Source ← I/O Device

 Destination ← Memory

 Transfer Mode ← DMA

 Transfer Size ← Required Data Size

 Enable DMA Interrupt

Start I/O Device

Start DMA Transfer

WHILE DMA Transfer is not completed DO

 CPU performs other tasks

END WHILE

IF DMA Transfer Complete Interrupt occurs THEN

 Save CPU state

 Identify DMA Interrupt

 Clear DMA Interrupt Flag

 Check Transfer Status

 IF Transfer is successful THEN

 Set Transfer Status ← COMPLETED

 ELSE

 Set Transfer Status ← ERROR

 END IF

 Restore CPU state

 Resume CPU execution

END IF

Disable DMA Controller

END

2. Develop pseudocode for an interrupt handling system that prioritizes vectored interrupts from multiple devices and dispatches appropriate service routines.

Ans:

ALGORITHM Vectored_Interrupt_Handler

BEGIN

 Initialize Interrupt Controller

 Define Interrupt Sources:

 Device 1

 Device 2

 Device 3

 Device 4

Assign Priority:

Device 1 $\leftarrow$ Highest Priority

Device 2 $\leftarrow$ High Priority

Device 3 $\leftarrow$ Medium Priority

Device 4 $\leftarrow$ Low Priority

Enable Interrupts

WHILE System is Running DO

IF Interrupt Request is received THEN

Save CPU state

Identify interrupting device

Determine interrupt vector

Compare priorities of pending interrupts

Select highest-priority interrupt

Jump to corresponding Interrupt Service Routine (ISR)

Execute selected ISR

Clear interrupt request

Restore CPU state

Return to interrupted program

END IF

END WHILE

END

3. Write pseudocode to select between memory-mapped I/O and I/O-mapped I/O dynamically based on latency and bandwidth requirements.

Ans:

ALGORITHM Dynamic_IO_Mode_Selection

BEGIN

Read I/O Device Requirements

Input Latency Requirement

Input Bandwidth Requirement

Set Latency_Threshold

Set Bandwidth_Threshold

IF Latency Requirement is HIGH AND
Bandwidth Requirement is HIGH THEN

Select I/O Mode $\leftarrow$ Memory-Mapped I/O

ELSE IF Latency Requirement is LOW AND
Bandwidth Requirement is LOW THEN

Select I/O Mode $\leftarrow$ I/O-Mapped I/O

ELSE

Calculate Memory-Mapped I/O Performance

Calculate I/O-Mapped I/O Performance

IF Memory-Mapped I/O Performance >
I/O-Mapped I/O Performance THEN

Select I/O Mode $\leftarrow$ Memory-Mapped I/O

ELSE

Select I/O Mode $\leftarrow$ I/O-Mapped I/O

END IF

END IF

Configure I/O Controller using Selected I/O Mode

Start Data Transfer

Monitor Latency and Bandwidth

```
IF Performance becomes unsuitable THEN
    Recalculate Requirements
    Select Appropriate I/O Mode
END IF
```

END

4. Design pseudocode for a bus arbitration algorithm that manages multiple devices communicating over PCIe, USB, and Thunderbolt interfaces.

Ans:

ALGORITHM Bus_Arbitration

BEGIN

Initialize PCIe, USB and Thunderbolt Interfaces

Create Device Request Queue

WHILE System is Running DO

 Check all interfaces for pending requests

 IF PCIe Device requests bus access THEN

 Add PCIe request to Request Queue

 END IF

 IF USB Device requests bus access THEN

 Add USB request to Request Queue

 END IF

 IF Thunderbolt Device requests bus access THEN

 Add Thunderbolt request to Request Queue

 END IF

 Assign Priority to each request

 Priority depends on:

 Data Size

 Latency Requirement

 Device Type

 Request Waiting Time

 Select highest-priority request

IF Bus is Available THEN

Grant Bus Access to selected device

Perform Data Transfer

Monitor Transfer Status

IF Transfer is Completed THEN

Release Bus

Remove request from Request Queue

END IF

ELSE

Keep request in Request Queue

Wait until Bus becomes available

END IF

Increase waiting time of pending requests

IF a request waits for too long THEN

Increase its priority

END IF

END WHILE

END

5. Write pseudocode to implement a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupts for high-speed devices.

Ans:

ALGORITHM Hybrid_IO_Communication

BEGIN

Initialize I/O Devices

FOR each connected device DO

Determine Device Speed

IF Device Speed = LOW THEN

Select I/O Method $\leftarrow$ Polling

WHILE Device is Active DO

 Check Device Status

 IF Data is Available THEN

 Read Data from Device

 Store Data in Memory

 END IF

END WHILE

ELSE IF Device Speed = HIGH THEN

 Select I/O Method $\leftarrow$ DMA with Interrupts

 Configure DMA Controller

 Set Source $\leftarrow$ High-Speed I/O Device

 Set Destination $\leftarrow$ Memory

 Set Transfer Size

 Enable DMA Completion Interrupt

 Start DMA Transfer

 CPU performs other tasks

 IF DMA Completion Interrupt occurs THEN

 Save CPU state

 Check DMA Transfer Status

 IF Transfer is Successful THEN

 Update Data Transfer Status

 ELSE

 Handle Transfer Error

 END IF

 Clear Interrupt

Restore CPU state

Resume CPU execution

END IF

END IF

END FOR

END

Code of Conduct:

I ____Bhoomika (192524291)____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Problem Understanding (20%)	Algorithm Design (30%)	Use of Control Structures (20%)	Pseudocode Structure (10%)	Completeness (20%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
----------	-----------	---------------------	----------------------	----------------------	---------------------

Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient